

## 4.3 - Criptografia em repouso e envelope encryption

Três camadas de defesa que não se substituem

# A auditoria que travou um contrato

- **Cliente farmacêutico audita a infraestrutura da LunaLog**
  - Transporte de medicamentos controlados exige compliance rígido

## LUNALOG

Em 2023, o consultor de segurança enviado por um cliente farmacêutico auditou a infraestrutura da LunaLog e encontrou um problema grave: os dados de localização e rota de entrega de medicamentos controlados não estavam criptografados em repouso no banco. Sem esse requisito atendido, o contrato de 800 mil reais anuais simplesmente não seria assinado. A decisão antiga de não criptografar dados em repouso, tomada para simplificar a operação, acabou custando muito mais do que o investimento em criptografia teria custado. Roberto entendeu ali que repouso não era detalhe, era pré-requisito comercial.

# Três lugares onde a criptografia vive

Em trânsito, em repouso e em uso

---

- **Em trânsito protege o dado enquanto ele se move**
  - TLS e mTLS cobrem essa camada na rede
- **Em repouso protege o dado enquanto ele descansa**
  - Disco, volume, banco e object storage cifrados
- **Em uso é a fronteira mais difícil, dado em memória**
  - Confidential computing e enclaves endereçam esse caso
- **As três camadas são independentes, não substitutas**

# Onde cifrar dados em repouso

## Cifragem de volume ou disco

- Transparente para a aplicação
- Protege contra roubo físico de mídia
- Não protege contra acesso lógico ao banco
- Granularidade grossa, tudo ou nada

## Cifragem de campo na aplicação

- Protege campos sensíveis individualmente
- Sobrevive a acesso direto ao banco
- Exige gestão de chave na aplicação
- Granularidade fina, custo de desenvolvimento

# Envelope encryption com DEK e KEK

## PSEUDOCODE

```
# Cifrar um dado
DEK = gerar_chave_aleatoria()      # data key, local
dado_cifrado = AES_encrypt(dado, DEK)

# Proteger a DEK com a KEK do KMS
DEK_cifrada = KMS.encrypt(KEK_id, DEK)

# Armazenar dado e DEK cifrada juntos
salvar(dado_cifrado, DEK_cifrada)

# Decifrar quando preciso
DEK = KMS.decrypt(DEK_cifrada)
dado = AES_decrypt(dado_cifrado, DEK)
```

# Por que envelope encryption resolve rotação

- A DEK cifra o dado, a KEK cifra a DEK
- Só a DEK criptografada fica junto do dado

Para rotacionar a chave mestra, basta recriptografar as DEKs, não os dados. Uma rotação que levaria horas passa a levar milissegundos.

- A KEK nunca sai do KMS ou do HSM em texto claro



Você tem criptografia em trânsito, em repouso e um padrão elegante de envelope. Mas tudo isso depende de uma única coisa: as chaves. E se a chave que protege anos de dados vazar num commit accidental? Quem controla o ciclo de vida dela, e como você rotaciona sem parar o sistema?

### **Gerenciamento e rotação de chaves em produção**